

N-body with Barnes-Hut Algorithm

B1228021 邱庭俞

一、專案構想與問題描述

N-body 問題問的是給 N 個有質量的粒子，每個粒子對其他所有粒子施加萬有引力，那要如何追蹤它們在時間中的運動軌跡？這個問題出現在天文模擬、分子動力學、電漿物理等領域，是計算物理的經典問題之一。

那最暴力解法就是每步都對所有粒子對算一次引力，複雜度是 $O(N^2)$ 。 $N=10,000$ 時每步就要做超過一億次浮點運算， $N=100,000$ 根本無法在合理時間內完成。

1.1 Barnes-Hut Algorithm

我選用 Barnes-Hut Algorithm，因為可以將每步的計算複雜度降到 $O(N \log N)$ 。因為距離夠遠的粒子群，可以用質心來代替整群計算引力，而不需要每個粒子累加。

具體做法就是以遞迴方式把二維空間切成四元樹，每個節點記錄覆蓋區域的總質量與質心。計算粒子 i 所受的力時，對樹的每個節點判斷。若節點邊長 s 與節點-粒子距離 d 的比值 $s/d < \theta$ ，直接用節點的質心替代，不再遞迴展開，否則繼續往子節點走。

1.2 課堂技術對應

使用 9 項平行化 API 且皆為課堂涵蓋技術，大於 50% 的規定。

技術/API	課堂對應主題	使用版本
MPI_Bcast	MPI	mpi.c / mpi_omp.c / master_worker.c
MPI_Allgatherv		mpi.c / mpi_omp.c
MPI_Send / MPI_Recv		master_worker.c
MPI_Reduce		全部 MPI 版本（取最大 comm_time）
MPI_Init_thread		mpi_omp.c / master_worker.c
MPI_Barrier / MPI_Wtime		全部 MPI 版本
#pragma omp parallel for	OpenMP	omp.c / mpi_omp.c / master_worker.c
schedule(dynamic)		omp.c / mpi_omp.c
Master-Worker Pattern	Master-Worker	master_worker.c

二、平行化的優勢

Barnes-Hut 最適合平行化的地方是力計算的主迴圈。每個粒子 i 的受力計算只需要讀取整棵 Quadtree，而寫入的目標只有自己的 f_x 、 f_y ，不同粒子之間完全沒有資料相依性。

實測 $N=10,000$ 、使用 4 processes + 4 threads 的 MPI+OMP 版本，在 10 個 simulation steps 中從 0.601 秒降到 0.096 秒，加速比 $6.27\times$ 。

除了速度，平行化也讓更大規模的模擬變得可行。 $N=50,000$ 的 serial 版本跑 10 steps 需要 4.05 秒，MPI+OMP 只需 0.80 秒，讓長時間、大粒子數的模擬從等到天荒地老變成一眨眼的事。

三、平行技術說明與選擇理由

3.1 Serial (基準版本)

單一 CPU、單一執行緒，依序執行：建樹 → 計算質量分佈 → 力計算 → Euler 積分 → 釋放樹。用 `clock_gettime(CLOCK_MONOTONIC)` 計時。這個版本的意義是提供正確性驗證的參考答案，以及計算其他版本加速比的分母。

3.2 OpenMP

OpenMP 最自然的切入點就是力計算迴圈。選 `schedule(dynamic)` 而非 `schedule(static)` 的原因是 Barnes-Hut 的工作量跟粒子的空間位置有關，靠近密集區域的粒子要展開更多樹節點，工作量比邊緣粒子多。Dynamic 排程讓忙完的執行緒繼續搶新工作，避免 static 切分下某些執行緒提早閒置的問題。chunk=32 在排程 overhead 與動態靈活性之間取得平衡。

3.3 MPI

MPI 把 N 個粒子按 rank 靜態切分，每個 rank 計算自己那一段粒子的受力與位置更新。每步的通訊流程：

MPI_Bcast：將完整的粒子陣列從 rank 0 廣播給所有 rank，確保每個 rank 都有最新狀態以建構完整的 Quadtree。

MPI_Allgatherv：力計算完成後，把各 rank 更新的那段粒子收集回來，讓每個 rank 的粒子陣列都是最新狀態。

每個 rank 建整棵樹（而非只建局部樹）是有意的設計。確保每個粒子都能看到全域的質量分佈，不會因為只有局部資訊而計算出錯誤的引力。代價是建樹工作重複 P 次，這個 tradeoff 在 N 較大時成為瓶頸。

3.4 MPI + OpenMP (混合平行)

這個版本把行程的 MPI 分割與執行緒的 OMP 疊加。MPI 決定每個 rank 負責哪一段粒子，OMP 在力計算迴圈中再把那段粒子分給 T 個執行緒。理論並行度達到 $P \times T = 16$ 。

MPI_Init_thread 使用 MPI_THREAD_FUNNELED，只有主執行緒 (rank 的主執行緒) 發起 MPI 呼叫，OMP 執行緒只做計算不碰 MPI。這樣不需要完全 thread-safe 的 MPI 實作，相容性與效能都比 MPI_THREAD_MULTIPLE 好。

3.5 Master-Worker

靜態切分在粒子分佈不均勻時容易產生負載失衡。Master-Worker 模式的設計邏輯是把工作切成更細的 chunk，誰做完誰就去領新工作，來達到動態負載平衡。

Rank 0 擔任 master，不參與計算，專門負責任務分派 (MPI_Send task) 和結果收集 (MPI_Recv result)。Rank 1~3 擔任 worker，在每個 chunk 內用 OMP 並行力計算，完成後立刻通報 master 並領取下一個 chunk。每步結束後，rank 0 用 MPI_Bcast 把更新後的完整粒子陣列廣播給所有 worker 供下一步使用。

計時設計：worker 的 compute_time 只累計 tree build + OMP force loop 的時間，MPI_Reduce(MAX) 取最慢的 worker 作為代表，Comm = Total - max(worker compute)。這樣才能正確反映 master-worker 架構下計算與通訊的真實數據。

四、效能評估

4.1 正確性驗證

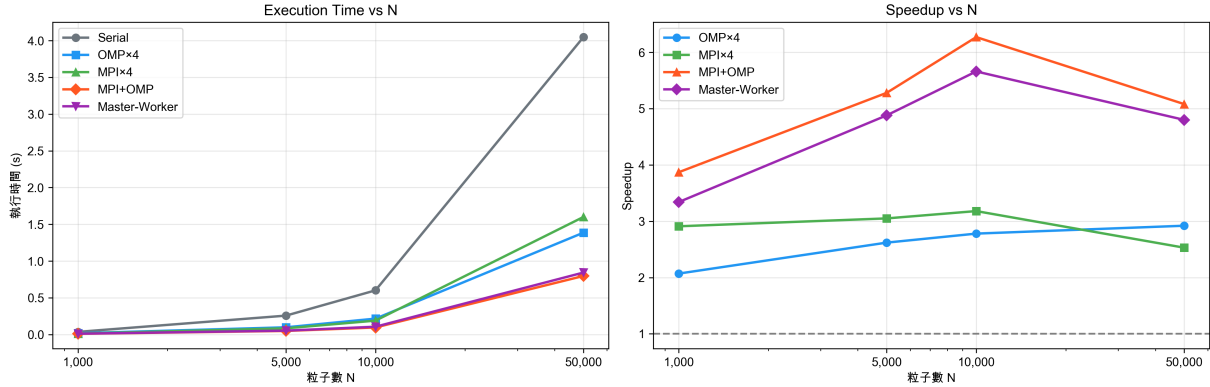
所有版本在每次模擬結束後計算所有粒子 x、y 座標的總和作為 checksum，並與 serial 版本對照。若 checksum 完全吻合 (到 7 位有效數字)，視為正確性驗證通過。

N	sum_x	sum_y	所有版本一致？
1,000	6.257054×10^{12}	-1.185071×10^{13}	✓ 通過
5,000	5.425829×10^{12}	-9.495344×10^{12}	✓ 通過
10,000	1.067059×10^{13}	-2.072669×10^{13}	✓ 通過
50,000	-2.175913×10^{14}	6.975083×10^{13}	✓ 通過

4.2 實驗一：N Scaling

固定 4 processes、4 threads、10 steps，改變粒子數 N，觀察各版本的擴展性。

N=1,000	
	<pre>--- N=1000 --- [Serial] N=1000 Steps=10 Threads=1 Compute=0.0364 s Comm=0.0000 s Total=0.0364 s [Verify] sum_x=6.257054e+12 sum_y=-1.185071e+13 [OMP] N=1000 Steps=10 Threads=4 Compute=0.0176 s Comm=0.0000 s Total=0.0176 s [Verify] sum_x=6.257054e+12 sum_y=-1.185071e+13 --> OMP(T=4) Speedup= 2.07x Efficiency= 51.7% [MPI] N=1000 Steps=10 Procs=4 Compute=0.0113 s Comm=0.0013 s Total=0.0125 s [Verify] sum_x=6.257054e+12 sum_y=-1.185071e+13 --> MPI(P=4) Speedup= 2.91x Efficiency= 72.8% [MPI+OMP] N=1000 Steps=10 Procs=4 Threads=4 Compute=0.0082 s Comm=0.0012 s Total=0.0094 s [Verify] sum_x=6.257054e+12 sum_y=-1.185071e+13 --> MPI+OMP(P=4,T=4) Speedup= 3.87x Efficiency= 24.2% [Master-Worker] N=1000 Steps=10 Procs=4 Threads=4 Chunk=50 Compute=0.0082 s Comm=0.0026 s Total=0.0109 s [Verify] sum_x=6.257054e+12 sum_y=-1.185071e+13 --> MasterWorker(P=4,T=4) Speedup= 3.34x Efficiency= 27.8%</pre>
N=5,000	
	<pre>--- N=5000 --- [Serial] N=5000 Steps=10 Threads=1 Compute=0.2568 s Comm=0.0000 s Total=0.2568 s [Verify] sum_x=5.425829e+12 sum_y=-9.495344e+12 [OMP] N=5000 Steps=10 Threads=4 Compute=0.0981 s Comm=0.0000 s Total=0.0981 s [Verify] sum_x=5.425829e+12 sum_y=-9.495344e+12 --> OMP(T=4) Speedup= 2.62x Efficiency= 65.4% [MPI] N=5000 Steps=10 Procs=4 Compute=0.0816 s Comm=0.0026 s Total=0.0842 s [Verify] sum_x=5.425829e+12 sum_y=-9.495344e+12 --> MPI(P=4) Speedup= 3.05x Efficiency= 76.2% [MPI+OMP] N=5000 Steps=10 Procs=4 Threads=4 Compute=0.0424 s Comm=0.0062 s Total=0.0486 s [Verify] sum_x=5.425829e+12 sum_y=-9.495344e+12 --> MPI+OMP(P=4,T=4) Speedup= 5.28x Efficiency= 33.0% [Master-Worker] N=5000 Steps=10 Procs=4 Threads=4 Chunk=250 Compute=0.0436 s Comm=0.0090 s Total=0.0526 s [Verify] sum_x=5.425829e+12 sum_y=-9.495344e+12 --> MasterWorker(P=4,T=4) Speedup= 4.88x Efficiency= 40.7%</pre>
N=10,000	
	<pre>--- N=10000 --- [Serial] N=10000 Steps=10 Threads=1 Compute=0.6009 s Comm=0.0000 s Total=0.6009 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 [OMP] N=10000 Steps=10 Threads=4 Compute=0.2163 s Comm=0.0000 s Total=0.2163 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> OMP(T=4) Speedup= 2.78x Efficiency= 69.5% [MPI] N=10000 Steps=10 Procs=4 Compute=0.1858 s Comm=0.0029 s Total=0.1888 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI(P=4) Speedup= 3.18x Efficiency= 79.6% [MPI+OMP] N=10000 Steps=10 Procs=4 Threads=4 Compute=0.0915 s Comm=0.0044 s Total=0.0959 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=4,T=4) Speedup= 6.27x Efficiency= 39.2% [Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=500 Compute=0.0895 s Comm=0.0166 s Total=0.1062 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(P=4,T=4) Speedup= 5.66x Efficiency= 47.2%</pre>
N=50,000	
	<pre>--- N=50000 --- [Serial] N=50000 Steps=10 Threads=1 Compute=4.0478 s Comm=0.0000 s Total=4.0478 s [Verify] sum_x=-2.175913e+14 sum_y=6.975083e+13 [OMP] N=50000 Steps=10 Threads=4 Compute=1.3871 s Comm=0.0000 s Total=1.3871 s [Verify] sum_x=-2.175913e+14 sum_y=6.975083e+13 --> OMP(T=4) Speedup= 2.92x Efficiency= 73.0% [MPI] N=50000 Steps=10 Procs=4 Compute=1.5751 s Comm=0.0233 s Total=1.5984 s [Verify] sum_x=-2.175913e+14 sum_y=6.975083e+13 --> MPI(P=4) Speedup= 2.53x Efficiency= 63.3% [MPI+OMP] N=50000 Steps=10 Procs=4 Threads=4 Compute=0.7656 s Comm=0.0318 s Total=0.7974 s [Verify] sum_x=-2.175913e+14 sum_y=6.975083e+13 --> MPI+OMP(P=4,T=4) Speedup= 5.08x Efficiency= 31.7% [Master-Worker] N=50000 Steps=10 Procs=4 Threads=4 Chunk=2500 Compute=0.7259 s Comm=0.1177 s Total=0.8436 s [Verify] sum_x=-2.175913e+14 sum_y=6.975083e+13 --> MasterWorker(P=4,T=4) Speedup= 4.80x Efficiency= 40.0%</pre>



N	Serial (s)	OMP×4	MPI×4	MPI+OMP	Master-Worker
1,000	0.036	0.018 (2.07×)	0.013 (2.91×)	0.009 (3.87×)	0.011 (3.34×)
5,000	0.257	0.098 (2.62×)	0.084 (3.05×)	0.049 (5.28×)	0.053 (4.88×)
10,000	0.601	0.216 (2.78×)	0.189 (3.18×)	0.096 (6.27×)	0.106 (5.66×)
50,000	4.048	1.387 (2.92×)	1.598 (2.53×)	0.797 (5.08×)	0.844 (4.80×)

N=1,000 時各方法的加速比偏低，因為計算量太小，平行化的啟動開銷（OMP fork/join、MPI Bcast 延遲）佔比較高。從 N=5,000 開始，計算量拉大後加速比都明顯改善。MPI+OMP 在 N=10,000 達到最高的 6.27×。

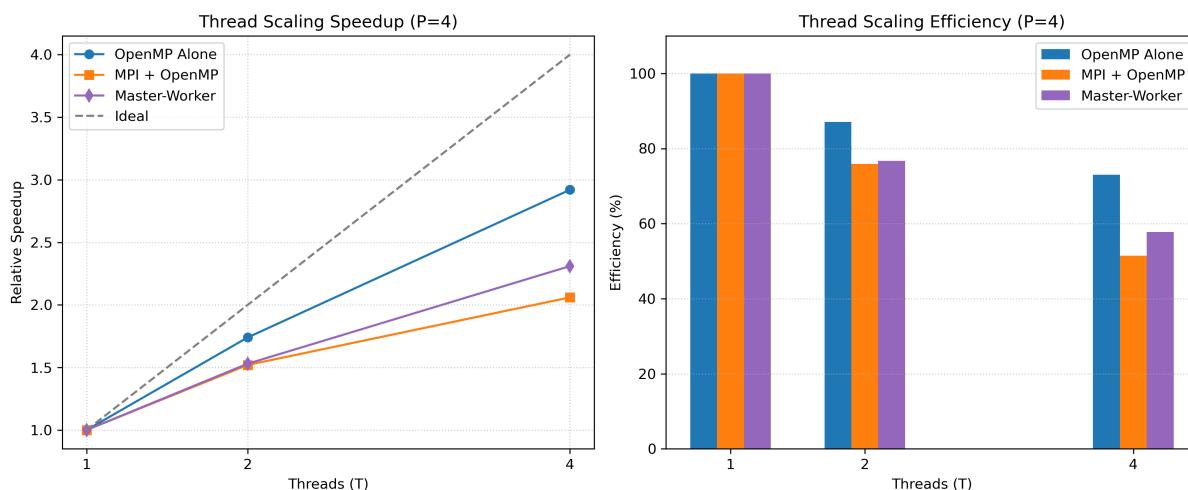
觀察到 N=50,000 的 MPI (2.53×) 比 OMP (2.92×) 還慢。這個版本的每個 rank 都要獨立建構完整的 50,000 粒子 Quadtree，4 個 rank 等於要建 4 遍樹。建樹本身的複雜度 $O(N \log N)$ 在 N=50,000 時已相當可觀，重複工作的代價超過了行程分工的收益。OMP 版本只建一棵樹，用 4 個執行緒加速力計算，沒有這個問題，所以更快。

4.3 實驗二：Thread Scaling

固定粒子數 N=10,000、固定行程數 P=4。以各版本自身 T=1 的執行時間為基準分母，來觀察純執行緒增長對效能的貢獻。

OMP_NUM_THREADS=1
<pre> --- OMP_NUM_THREADS=1 --- [OMP] N=10000 Steps=10 Threads=1 Compute=0.6189 s Comm=0.0000 s Total=0.6189 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> OMP(T=1) Speedup= 0.94x Efficiency= 93.9% [MPI+OMP] N=10000 Steps=10 Procs=4 Threads=1 Compute=0.1946 s Comm=0.0083 s Total=0.2030 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=4,T=1) Speedup= 2.86x Efficiency= 71.5% [Master-Worker] N=10000 Steps=10 Procs=4 Threads=1 Chunk=500 Compute=0.2332 s Comm=0.0145 s Total=0.2477 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(P=4,T=1) Speedup= 2.35x Efficiency= 78.2%</pre>
OMP_NUM_THREADS=2

<pre> --- OMP_NUM_THREADS=2 --- [OMP] N=10000 Steps=10 Threads=2 Compute=0.3552 s Comm=0.0000 s Total=0.3552 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> OMP(T=2) Speedup= 1.64x Efficiency= 81.8% [MPI+OMP] N=10000 Steps=10 Procs=4 Threads=2 Compute=0.1265 s Comm=0.0071 s Total=0.1337 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=4,T=2) Speedup= 4.34x Efficiency= 54.3% [Master-Worker] N=10000 Steps=10 Procs=4 Threads=2 Chunk=500 Compute=0.1464 s Comm=0.0150 s Total=0.1615 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(P=4,T=2) Speedup= 3.60x Efficiency= 59.9% </pre>	
OMP_NUM_THREADS=4	
<pre> --- OMP_NUM_THREADS=4 --- [OMP] N=10000 Steps=10 Threads=4 Compute=0.2119 s Comm=0.0000 s Total=0.2119 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> OMP(T=4) Speedup= 2.74x Efficiency= 68.5% [MPI+OMP] N=10000 Steps=10 Procs=4 Threads=4 Compute=0.0937 s Comm=0.0050 s Total=0.0987 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=4,T=4) Speedup= 5.89x Efficiency= 36.8% [Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=500 Compute=0.0899 s Comm=0.0173 s Total=0.1072 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(P=4,T=4) Speedup= 5.42x Efficiency= 45.2% </pre>	



Threads (T)	OMP	效率	MPI+OMP	效率	Master-Worker	效率
1	0.6189s (1.00×)	100.0%	0.203s (1.00×)	100.0%	0.248s (1.00×)	100.0%
2	0.355s (1.74×)	87.1%	0.134s (1.52×)	75.9%	0.162s (1.53×)	76.7%
4	0.212s (2.92×)	73.0%	0.099s (2.06×)	51.4%	0.107s (2.31×)	57.8%

從結果可以看出，OpenMP 在執行緒數從 1 增加到 4 時，執行時間由 0.6189 秒降至 0.2119 秒，但效率維持在約 73%。效率無法持續提升的主要原因，在於 Barnes-Hut 演算法中的 Quadtree 建樹階段屬於序列工作，無法完全平行化，再加上執行緒之間的同步開銷，限制了整體加速效果。

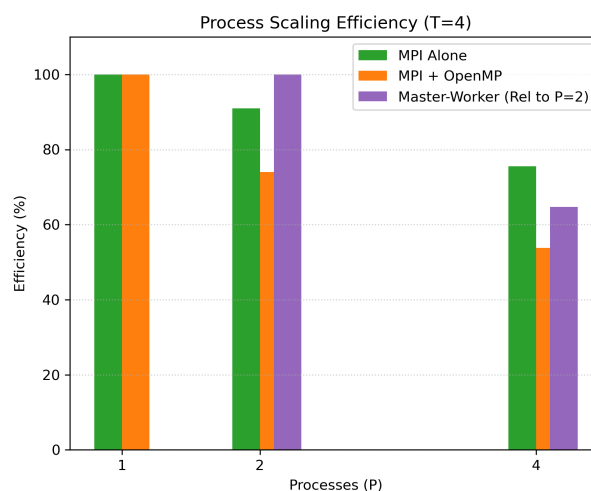
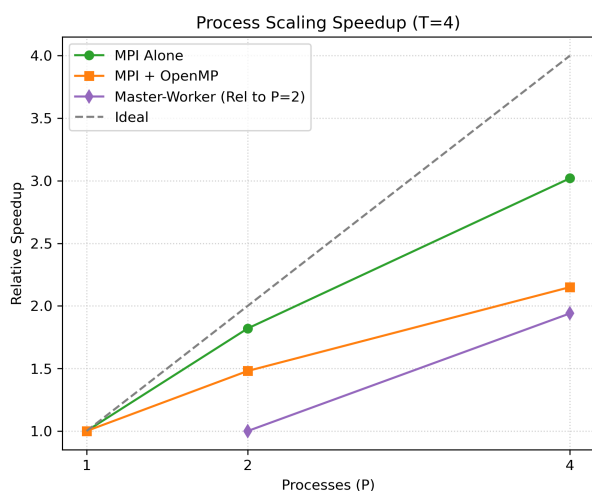
另一方面，在 MPI+OMP 混合模式下，當執行緒數增加到 4 時，執行緒效率下降至 51.4%。這是因為在 4 個 MPI 行程的情況下，每個行程分配到的粒子數已被分

散，導致單一任務的計算時間縮短（由 0.2030 秒降至 0.0987 秒），使 OpenMP 的 Fork/Join 固定開銷所佔比例提高，進而造成邊際效益遞減。相比之下，Master-Worker 模式在 4 個執行緒時仍能維持 57.8% 的效率，高於 MPI+OMP 的 51.4%，顯示動態任務分配機制能有效提升負載平衡，讓各 Worker 內部的執行緒保持較高利用率，減少閒置等待時間，因此展現出較佳的執行緒擴展性。

4.4 實驗三：Process Scaling

固定粒子數 $N=10,000$ 、固定執行緒數 $T=4$ 。以各版本自身 $P=1$ 的執行時間為基準分母，觀察多行程分工與通訊開銷對擴展性的影響。

P=1	
<pre>[MPI] N=10000 Steps=10 Procs=1 Compute=0.5804 s Comm=0.0000 s Total=0.5804 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI(P=1) Speedup= 1.01x Efficiency=101.1% [MPI+OMP] N=10000 Steps=10 Procs=1 Threads=4 Compute=0.2067 s Comm=0.0000 s Total=0.2067 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=1,T=4) Speedup= 2.84x Efficiency= 70.9%</pre>	
P=2	
<pre>[MPI] N=10000 Steps=10 Procs=2 Compute=0.3163 s Comm=0.0029 s Total=0.3193 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI(P=2) Speedup= 1.84x Efficiency= 91.8% [MPI+OMP] N=10000 Steps=10 Procs=2 Threads=4 Compute=0.1298 s Comm=0.0099 s Total=0.1397 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=2,T=4) Speedup= 4.20x Efficiency= 52.5% [Master-Worker] N=10000 Steps=10 Procs=2 Threads=4 Chunk=500 Compute=0.1931 s Comm=0.0187 s Total=0.2118 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(P=2,T=4) Speedup= 2.77x Efficiency= 69.2%</pre>	
P=4	
<pre>[MPI] N=10000 Steps=10 Procs=4 Compute=0.1868 s Comm=0.0052 s Total=0.1921 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI(P=4) Speedup= 3.05x Efficiency= 76.3% [MPI+OMP] N=10000 Steps=10 Procs=4 Threads=4 Compute=0.0918 s Comm=0.0045 s Total=0.0963 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MPI+OMP(P=4,T=4) Speedup= 6.09x Efficiency= 38.1% [Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=500 Compute=0.0924 s Comm=0.0166 s Total=0.1090 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(P=4,T=4) Speedup= 5.38x Efficiency= 44.8%</pre>	



Procs (P)	MPI	效率	MPI+OMP	效率
1	0.580s (1.00×)	100.0%	0.207s (1.00×)	100.0%
2	0.319s (1.82×)	91.0%	0.140s (1.48×)	74.0%
4	0.192s (3.02×)	75.5%	0.096s (2.15×)	53.8%

由於 Master-Worker 架構至少需要 1 個 Master 與 1 個 Worker，所以必須從 P=2 開始執行，沒有 P=1 的基準數據可供比較，所以沒有納入 Process Scaling 的分析表格。如果單獨觀察其行程擴展表現，當行程數由 P=2（0.2118 秒）增加至 P=4（0.1090 秒）時，Worker 節點數從 1 個增加到 3 個，擴展了 3 倍，但實際僅獲得 1.94 倍加速，比較後的行程利用效率約為 64.7%。

另一方面，純 MPI 在 P=2 時仍能維持 91.0% 的高效率，主要因為通訊成本相當低，然而當行程增加至 P=4 時，效率下降至 75.5%。造成這種現象的原因是因為採用全域建樹的設計，每個新增的 Rank 都必須在本地端重新建立一棵完整的 N=10,000 粒子 Quadtree，使得這部分 $O(N \log N)$ 的計算工作無法隨行程數增加而被分攤，逐漸成為限制擴展性的主要瓶頸。相較之下，MPI+OMP 在 P=4 時的行程擴展效率更進一步下降至 53.8%，雖然執行時間從 0.2067 秒縮短至 0.0963 秒，但當行程數與執行緒數同步增加時，MPI_Bcast、MPI_Allgather 等集體通訊所帶來的網路延遲，以及行程之間的同步等待時間，會相互疊加並放大其影響，導致整體擴展效率明顯低於純 MPI 架構。

4.5 實驗四：Chunk Size Effect

固定 N=10,000、4 processes、4 threads、10 steps，改變 chunk 大小，找出最佳的動態任務粒度。Serial baseline = 0.586s。

Serial
<pre>[Serial] N=10000 Steps=10 Threads=1 Compute=0.5863 s Comm=0.0000 s Total=0.5863 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13</pre>
Chunk=10
<pre>--- Chunk=10 --- [Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=10 Compute=0.1995 s Comm=0.0193 s Total=0.2188 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(chunk=10) Speedup= 2.68x Efficiency= 22.3%</pre>
Chunk=50
<pre>--- Chunk=50 --- [Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=50 Compute=0.1078 s Comm=0.0176 s Total=0.1254 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(chunk=50) Speedup= 4.68x Efficiency= 39.0%</pre>
Chunk=100
<pre>--- Chunk=100 --- [Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=100 Compute=0.0963 s Comm=0.0173 s Total=0.1136 s [Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13 --> MasterWorker(chunk=100) Speedup= 5.16x Efficiency= 43.0%</pre>
Chunk=200


```

--- Chunk=200 ---
[Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=200 Compute=0.0969 s Comm=0.0168 s Total=0.1137 s
[Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13
--> MasterWorker(chunk=200) Speedup= 5.16x Efficiency= 43.0%

```

Chunk=500

```

--- Chunk=500 ---
[Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=500 Compute=0.0901 s Comm=0.0171 s Total=0.1073 s
[Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13
--> MasterWorker(chunk=500) Speedup= 5.46x Efficiency= 45.5%

```

Chunk=1,000

```

--- Chunk=1000 ---
[Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=1000 Compute=0.0910 s Comm=0.0175 s Total=0.1085 s
[Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13
--> MasterWorker(chunk=1000) Speedup= 5.40x Efficiency= 45.0%

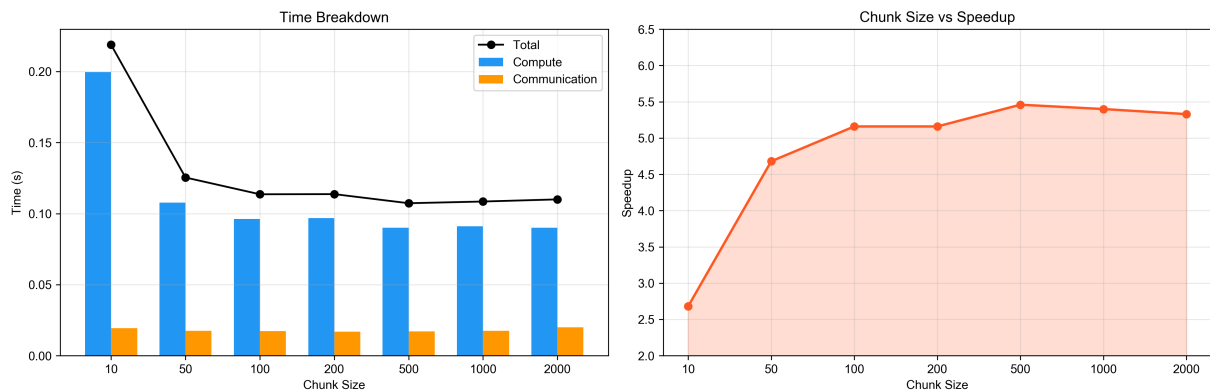
```

Chunk=2,000

```

--- Chunk=2000 ---
[Master-Worker] N=10000 Steps=10 Procs=4 Threads=4 Chunk=2000 Compute=0.0900 s Comm=0.0199 s Total=0.1100 s
[Verify] sum_x=1.067059e+13 sum_y=-2.072669e+13
--> MasterWorker(chunk=2000) Speedup= 5.33x Efficiency= 44.4%

```



Chunk	每步 Task 數	Compute (s)	Comm (s)	Total (s)	Speedup
10	1,000	0.1995	0.0193	0.2188	2.68×
50	200	0.1078	0.0176	0.1254	4.68×
100	100	0.0963	0.0173	0.1136	5.16×
200	50	0.0969	0.0168	0.1137	5.16×
500	20	0.0901	0.0171	0.1073	5.46×
1,000	10	0.0910	0.0175	0.1085	5.40×
2,000	5	0.0900	0.0199	0.1100	5.33×

Comm 時間在所有 chunk 大小下幾乎不變 (0.017~0.019s)，說明訊息本身的傳輸並不貴。影響效能的主因是 Compute 時間：

Chunk=10 的 Compute=0.1995s 是最大值：每個 task 只有 10 個粒子，#pragma omp parallel for schedule(dynamic, 8) 的 fork/join 和工作分配 overhead 比實際計算粒子引力的時間還多，OMP 在這麼少的迭代次數下根本沒有效益。

Chunk=2,000 的 Comm 略微上升至 0.0199s，Speedup 從 5.46× 降到 5.33×：每步只有 5 個 task，3 個 worker 完成第一輪後，尾端的 1~2 個 task 只有部分 worker 在跑，其餘閒

置，出現了尾端負載失衡。最佳任務力度出現在 $\text{Chunk}=500$ （每步 20 個 task），加速比最高的 $5.46\times$ 。而這時候每個 task 有 500 個粒子，OMP 的 4 個執行緒各處理 125 個粒子，足以有效攤銷 fork 開銷，且 20 個 task 讓 3 個 worker 有充分的動態平衡空間，不會有人提早閒置沒事做。

五、討論

5.1 MPI 大 N 效能反轉的原因

在粒子數達到 $N=50,000$ 的時候，MPI 的加速比只有 2.53 倍，表現輸給純 OpenMP 的 2.92 倍。這背後最根本的原因在於重複建樹。因為 Barnes-Hut 演算法每次模擬都需要重新建立四元樹，在 N 大到五萬時，光是建一棵樹就要花大約 0.2 秒。由於我們目前的 MPI 實作是讓 4 個 Rank 在各自的本地端各建一棵一模一樣的全域樹，算下來等於 4 個行程白白浪費了 0.8 秒在做重複的虛工，這幾乎快把平行計算省下來的時間給賠光了。這段重複建樹的時間會隨著行程數變多而等比例放大，嚴重拖累了擴展性。而 OpenMP 只需要在共享記憶體裡建一次樹，4 個執行緒就能專心去分配力計算的工作，在大 N 的時候自然比 MPI 更吃香。

5.2 MPI+OMP 效率為何看起來偏低

看到 MPI+OMP 在 $P=4, T=4$ 總共動用了 16 個計算單位時，全域效率只有 38.1%、加速比 6.09 倍，看似不怎麼樣，實際上，它已經成功把總時間從 0.587 秒直接壓到 0.096 秒，整整快了 6 倍多，對於一個有序列部分又包含通訊開銷的 $O(N \log N)$ 演算法來說，這其實是一個很不錯的結果。之所以效率數字看起來不好看，是因為碰到了強擴展性下的雙重瓶頸。在行程端，它依然擺脫不了多個 Rank 重重重複建全域樹的宿命，而在執行緒端，當 16 個並行單位同時去瓜分固定一萬個粒子時，每個人分到的工作量被稀釋，導致單一任務的計算時間縮短到只剩 0.09 秒左右。結果就是 OpenMP 執行緒每次 Fork/Join 的固定開銷，以及 MPI_Bcast 和 MPI_Allgather 的網路通訊延遲，佔總時間的比例反而被急遽放大，進而造成嚴重的邊際效益遞減。

5.3 Master-Worker 的定位

在粒子均勻隨機分佈的測試環境下，Master-Worker 的加速比為 5.66 倍（時間 0.106 秒），比靜態切分的 MPI+OMP 慢了大概 10%。這種劣勢其實在平行運算架構中是必然的，因為當大家的工作量本來就一樣忙的時候，靜態直接切塊的 MPI+OMP 效率幾乎是最優的，這時候 Master-Worker 為了動態派發和回收任務，還得頻繁透過 MPI_Send 和 MPI_Recv 來做點對點通訊，這些額外的傳輸時間在均勻分佈下完全沒有

對應的收益，全變成了效能淨損。不過，Master-Worker 的真正價值是要拿來對付空間分佈不均勻的真實場景。如果粒子大量聚集成幾群密集星系，導致大部分空間都是真空，靜態切分絕對會讓某些 Rank 忙死、某些閒死，這時候 Master-Worker 的動態分派就能發揮強大的適應性，讓手腳快的 Worker 自動去搶微調後的 Chunk 來做，進而大幅消弭行程間的閒置等待時間。

5.4 Chunk Size 對 OMP 效益的影響

實驗四的數據顯示，當 Chunk=10 的時候，計算時間高達 0.1995 秒，足足是 Chunk=500 時（0.0901 秒）的兩倍多。這說明了 Chunk 大小如果抓得太小，會直接搞垮工作包內部的 OpenMP 並行效益。當 Chunk 只有 10 的時候，代表一個任務包才包了 10 個粒子，此時底層的 `#pragma omp parallel for` 卻要把這 10 個粒子分配給 4 個執行緒去算，結果執行緒剛開好（Fork）就結束了（Join）。這種管理執行緒的同步與排程成本，遠遠高於實際算引力的實質工作量，讓平行化反而變成了負效益。相反，當換成 Chunk=500 時，4 個執行緒每人平均分到 125 個粒子，工作量才剛好符合，也才足以攤平執行緒的切換成本。所以在設計這種混合式 Master-Worker 機制時，Chunk 的下限至少要是總執行緒數的數十倍以上，底層的 OpenMP 才會真正發揮加速效果。

5.5 各平行模型之通訊開銷比較

平行模型 (N=50,000)	Compute(s)	Comm(s)	Total(s)
Serial	4.0478 s	0.0000 s	4.0478 s
OpenMP (T=4)	1.3871 s	0.0000 s	1.3871 s
MPI (P=4)	1.5751 s	0.0233 s	1.5984 s
MPI + OpenMP (P=4, T=4)	0.7656 s	0.0318 s	0.7974 s
Master-Worker (Chunk=2500)	0.7259 s	0.1177 s	0.8436 s

在 N=50,000 下，不同平行模型的通訊開銷呈現差異。實測數據顯示，MPI (P=4) 的通訊時間為 0.0233 秒，MPI+OMP (P=4, T=4) 為 0.0318 秒，而 Master-Worker (P=4, T=4, Chunk=2500) 的通訊時間則高達 0.1177 秒，是 MPI+OMP 的 3.5 至 5 倍。這樣的差異是來自於通訊模式的不同。MPI 與 MPI+OMP 每個 step 只需要執行一次集體的 MPI_Bcast 與 MPI_Allgather。而 Master-Worker 為了動態分配任務，必須頻繁依賴點對點的 MPI_Send 與 MPI_Recv 來派發與回收 Chunk。這也反映了 Master-Worker 架構需要取捨的內容，以較高的通訊延遲為代價，換取動態負載平衡的能力。在均勻分佈場景中，額外通訊開銷正是導致總執行時間略慢於靜態切分的 MPI+OMP 之主因。

六、AI 工具使用說明

使用 Claude Code 作為 AI 輔助工具，以下具體說明 AI 貢獻最大的部分：

使用情境	具體內容
Bug 修正	Master-Worker 的 active 計數器 deadlock bug： 當 $N < \text{chunk_size} \times (\text{size}-1)$ 時，部分 worker 收到終止訊號但不發送 TAG_DONE，master 永遠等待 → 死結。AI 協助分析邊界條件並修正計數邏輯。
計時架構設計	Master-Worker 的 compute/comm 分離：修正前 $\text{Compute} \approx 0$ （master 全程通訊， $\text{max_comm} \approx \text{elapsed}$ ）。AI 提出分開追蹤 worker compute_time，再用 MPI_Reduce(MAX) 取 bottleneck worker 的方案。
Job Script	四個實驗的設計架構、get_total bash 函式、print_perf 即時計算加速比/效率的邏輯。
核心設計	Barnes-Hut 演算法邏輯、Quadtree 資料結構。

七、結論

總共實作了五個 N-body Barnes-Hut 模擬版本，正確性全部通過 checksum 驗證，效能分析如下：

版本	最佳加速比 (N=10,000)	適合場景
Serial	1× (基準)	小 N 驗證、正確性參考
OpenMP	2.78× (T=4)	單機多核、均勻負載、簡單部署
MPI	3.18× (P=4)	中等 N 多行程分工，大 N 受建樹冗餘限制
MPI+OMP	6.27× (P=4, T=4)	最佳綜合效能，充分利用行程與執行緒
Master-Worker	5.66× (P=4, T=4)	動態負載平衡，粒子分佈不均時優勢更大

MPI+OMP 混合版本在 N=10,000 的環境下表現最佳（6.27×），適合作為首選。Master-Worker 雖然在均勻負載下慢，但在動態不均勻負載的真實場景中會具有更好的適應性。

粒子初始化：srand(123)，位置均勻隨機分佈於 $[0, 1000]^2$ ，質量介於 $10^{20} \sim 10^{24} + 10^{20}$ kg。

模擬參數：dt=0.01， $\theta=0.5$ ， $G=6.67430 \times 10^{-11}$ ，Steps=10。